



Visão Computacional e Deep Learning

Ma. Cilene Renata Real
cilene.real@facens.br

-
- **Segmentação de imagens x. detecção de objetos x classificação de imagens**
 - **Data Augmentation (Aumento de Dados)**
 - **Quantização e otimização para dispositivos de borda**

Segmentação de imagens x. detecção de objetos x classificação de imagens

A **segmentação de imagens** representa uma evolução avançada da classificação de imagens e detecção de objetos.

A ***classificação de imagens*** aplica um rótulo de classe a uma imagem.

A ***detecção de objetos*** combina a classificação de imagens com a localização de objetos, gerando regiões retangulares, chamadas de "caixas delimitadoras", em que os objetos estão localizados

Embora a detecção de objetos possa classificar vários elementos dentro de uma imagem e aproximar a largura e altura de cada elemento, **ela não pode discernir limites ou formas precisas.**

Isso limita a capacidade dos modelos de detecção de objetos convencionais de delinear objetos unidos de perto com caixas delimitadoras sobrepostas.

Segmentação de imagens x. detecção de objetos x classificação de imagens

A ***segmentação de imagens*** processa dados visuais no nível de pixel, usando várias técnicas para anotar pixels individuais como pertencentes a uma classe ou instância específica.

Técnicas de segmentação de imagem "clássicas" determinam anotações por meio da análise de qualidades inerentes de cada pixel (chamadas "heurística"), como cor e intensidade, enquanto modelos de deep learning empregam redes neurais complexas para reconhecimento sofisticado de padrões.

As saídas dessa anotação são ***máscaras de segmentação***, representando o limite pixel a pixel específico e a forma de cada classe (normalmente correspondendo a diferentes objetos, funcionalidades ou regiões) na imagem.

Segmentação de imagens x. detecção de objetos x classificação de imagens

A segmentação de imagens pode ser categorizada como:

- segmentação semântica;
- segmentação de instâncias.

Classes semânticas: “coisas” e “elementos do ambiente”

A diferença entre cada tipo de tarefa de segmentação de imagem reside na forma como tratam classes semânticas: as categorias específicas às quais um determinado pixel pode ser determinado pertencer.

No jargão da visão computacional, existem dois tipos de classes semânticas.

Cada um se presta a diferentes técnicas para segmentação precisa e eficaz.

Classes semânticas: “coisas” e “elementos do ambiente”

Coisas:

Coisas são classes de objetos com formas características, como “carro”, “árvore” ou “pessoa”.

Normalmente, as coisas têm instâncias claramente definidas que são contáveis.

Elas têm relativamente pouca variação de tamanho de uma instância para outra, bem como partes constituintes distintas da coisa em si: por exemplo, todos os carros têm rodas, mas uma roda não é um carro.

Classes semânticas: “coisas” e “elementos do ambiente”

Elementos do ambiente:

Elementos do ambiente se refere a classes semânticas que são amorfas e altamente variáveis em tamanho, como "céu" ou "água" ou "grama".

Normalmente, os elementos do ambiente não têm instâncias individuais contáveis e claramente definidas.

Ao contrário das coisas, os elementos do ambiente não têm partes distintas: uma lâmina de grama e campo de grama são ambas igualmente "grama".

Classes semânticas: “coisas” e “elementos do ambiente”

Algumas classes, sob certas condições de imagens, podem ser *coisas* ou *elementos do ambiente*.

Por exemplo, um grande grupo de pessoas pode ser interpretado como várias "pessoas" (cada uma sendo uma *coisa* de forma distinta e contável) ou uma "multidão" singular, com forma amorfa.

Embora a maioria dos esforços de detecção de objetos se concentre principalmente em classes de objetos, é importante considerar que elementos como céu, paredes, pisos e solo compõem a maioria do nosso contexto visual.

Classes semânticas: “coisas” e “elementos do ambiente”

Algumas classes, sob certas condições de imagens, podem ser *coisas* ou *elementos do*

Os elementos do ambiente são pontos de dados essenciais para identificar objetos, e vice-versa: um objeto metálico na estrada geralmente é um carro; o fundo azul atrás de um barco provavelmente é água, enquanto o fundo azul atrás de um avião provavelmente é céu.

Isto é de particular importância para modelos de deep learning.

Segmentação Semântica

A segmentação semântica é o tipo mais simples de segmentação de imagem.

Um modelo de segmentação semântica atribui uma classe semântica a cada pixel, mas não produz qualquer outro contexto ou informação (como objetos).

A segmentação semântica trata todos os pixels como *elementos do ambiente*; ela não diferencia entre *elementos do ambiente* e *objetos*.

Por exemplo, um modelo de segmentação semântica treinado para identificar certas classes em uma rua da cidade produziria máscaras de segmentação indicando as fronteiras e contornos para cada classe relevante de objetos (como veículos ou postes de luz) e elementos do ambiente (como estradas e calçadas), mas não faria qualquer distinção entre (ou contagem do número de) múltiplas instâncias da mesma classe. Por exemplo, carros estacionados um na frente do outro podem ser simplesmente tratados como um longo segmento de “carro”.

Segmentação Semântica



(a) Source Image



(b) Semantic Segmantation

Segmentação de Instâncias

A segmentação de instâncias inverte as prioridades da segmentação semântica: enquanto os algoritmos de segmentação semântica preveem apenas a classificação semântica de cada pixel (sem considerar instâncias individuais), **a segmentação de instâncias delinea a forma exata de cada instância de objeto separadamente.**

A segmentação por instância isola *objetos de elementos do ambiente*, que ignora e, portanto, pode ser entendida como uma forma evoluída de detecção de objetos que produz uma máscara de segmentação precisa em vez de uma caixa delimitadora aproximada.

Segmentação de Instâncias

É uma tarefa mais difícil do que a segmentação semântica: mesmo quando os objetos da mesma classe estão tocando ou mesmo se sobrepondo, os modelos de segmentação de instância devem ser capazes de separar e determinar a forma de cada um, enquanto os modelos de segmentação semântica podem simplesmente agrupá-los.

Considere, por exemplo, como os dois modelos diferentes tratam os veículos estacionados nesta imagem de uma rua urbana.



Segmentação de Instâncias

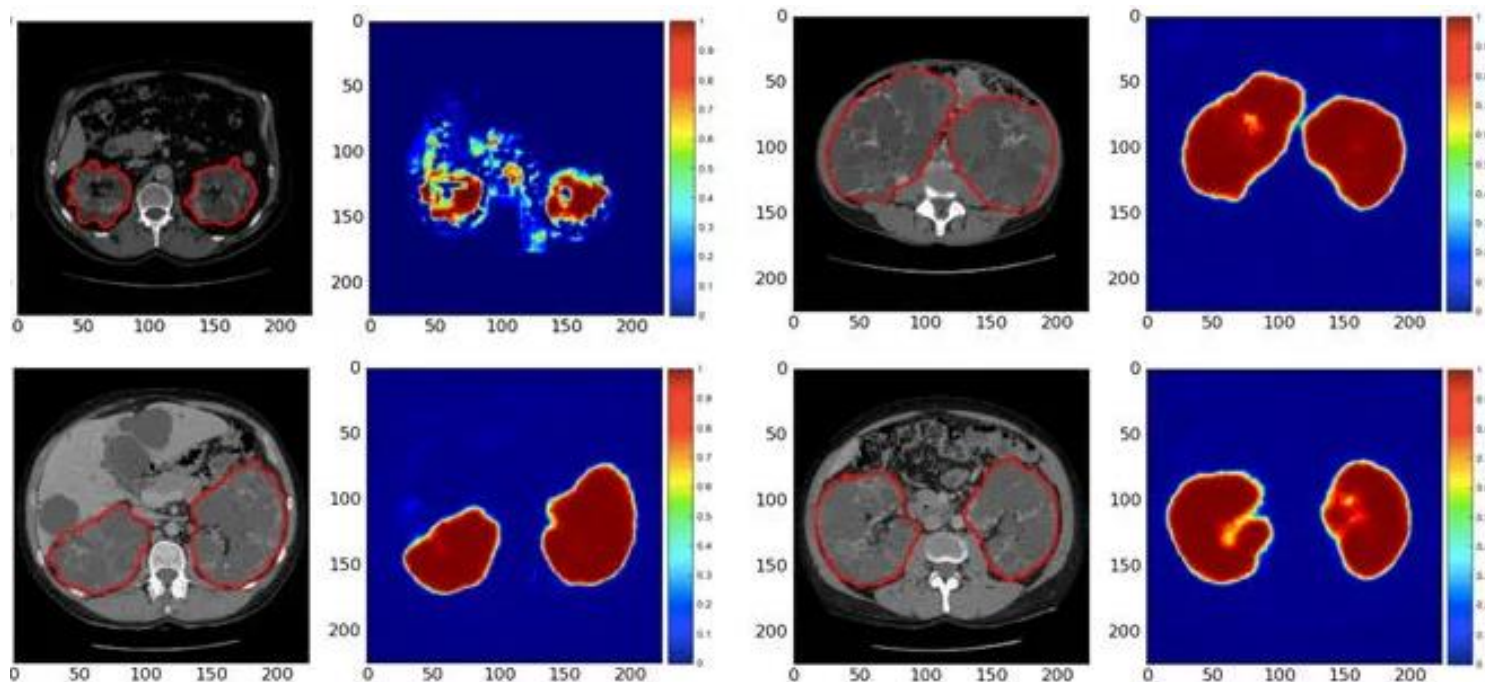
Os algoritmos de segmentação de instâncias geralmente adotam uma abordagem de dois estágios ou one-shot para o problema.

Modelos de dois estágios, como redes neurais convencionais (R-CNNs) baseadas na região, realizam a detecção de objetos convencionais para gerar caixas delimitadoras para cada instância proposta e, em seguida, realizam uma segmentação e classificação mais refinadas dentro de cada caixa delimitadora.

Modelos one-shot, como o YOLO (You Only Look Once), alcançam a segmentação de instâncias em tempo real ao realizar a detecção, classificação e segmentação de objetos simultaneamente.

Casos de uso de Segmentação de imagem

Imagens médicas: a segmentação de imagem possui muitas aplicações em radiografia, imagens de ressonância magnética (MRI), ultrassons e tomografia computadorizada (CT), auxiliando em tarefas como detecção de tumores, segmentação cerebral, diagnóstico de doenças e planejamento cirúrgico.



Casos de uso de Segmentação de imagem

Veículos autônomos: a segmentação de imagem permite que carros autônomos evitem obstáculos como pedestres e outros carros, além de identificar faixas e sinais de trânsito.

É usado da mesma forma para informar a navegação em robótica.

Imagem de satélite: a segmentação semântica e de instância automatizam a identificação de diferentes terrenos e feições topográficas.

Casos de uso de Segmentação de imagem

Cidades inteligentes: a segmentação de imagens alimenta tarefas como monitoramento e vigilância de tráfego em tempo real.

Fabricação: além de alimentar tarefas robóticas, a segmentação de imagens alimenta a classificação do produto e a detecção de defeitos.

Agricultura: a segmentação de imagens ajuda os agricultores a estimar os rendimentos das culturas e detectar ervas daninhas para remoção.

Segmentação de imagem na prática

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
img = cv2.imread('/content/drive/MyDrive/Visao/planta.jpg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
# Converter para HSV e segmentar verde (folha)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lower_green = np.array([35, 40, 40])
upper_green = np.array([85, 255, 255])
mask = cv2.inRange(hsv, lower_green, upper_green)
```

```
# Limpar ruído
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5,5), np.uint8))
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, np.ones((5,5), np.uint8))
```

```
# Encontrar contornos (separa folhas individuais)
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(img_rgb, contours, -1, (0, 255, 0), 2)
```

**Método estatístico,
sem utilizar nenhum
modelo.**

Segmentação de imagem na prática

```
# Visualizar
```

```
plt.figure(figsize=(15, 5))
```

```
plt.subplot(1, 3, 1)
```

```
plt.title("Imagem Original")
```

```
plt.imshow(img_rgb)
```

```
plt.axis('off')
```

```
plt.subplot(1, 3, 2)
```

```
plt.title("Máscara Verde (Folha)")
```

```
plt.imshow(mask, cmap='gray')
```

```
plt.axis('off')
```

```
plt.subplot(1, 3, 3)
```

```
plt.title("Contornos Detectados")
```

```
plt.imshow(img_rgb)
```

```
plt.axis('off')
```

```
plt.show()
```

```
# Estatística simples
```

```
area_folha = np.sum(mask > 0) / (mask.shape[0] *
```

```
mask.shape[1]) * 100
```

```
print(f"Área foliar estimada: {area_folha:.1f}% da imagem")
```

Segmentação com YOLO:

```
results = model.predict(  
    source="/content/drive/MyDrive/Visao/planta.jpg",  
    conf=0.25,  
    save=False  
)
```

Exibindo o resultado:

```
import matplotlib.pyplot as plt  
  
result = results[0]  
  
img_segmentada = result.plot()  
  
plt.figure(figsize=(10,8))  
plt.imshow(img_segmentada[...,:-1])  
plt.axis("off")  
plt.show()
```

Segmentação de imagem na prática

Agora, por gentileza, execute o código do "**Método estatístico, sem utilizar nenhum modelo**", para a imagem cachorro.jpg

Ele detectou algo verde na imagem?

Segmentação de imagem na prática

Execute o mesmo código para a imagem: floresta.jpg

E avaliem o que ele detectou?

Segmentação de imagem na prática

E o resultado foi:

Área foliar estimada: 0.0% da imagem

Porque esse código **não está detectando objetos**; ele está apenas **segmentando pixels cuja cor está dentro da faixa de verde no espaço**.

Segmentação de imagem na prática

Insira um novo código:

```
# Converter para HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Vermelho (faixa 1)
lower_red1 = np.array([0, 100, 100])
upper_red1 = np.array([10, 255, 255])

# Vermelho (faixa 2)
lower_red2 = np.array([170, 100, 100])
upper_red2 = np.array([180, 255, 255])

# Criar máscaras
mask1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask2 = cv2.inRange(hsv, lower_red2, upper_red2)

# Unir as duas máscaras
mask = cv2.bitwise_or(mask1, mask2)
```

Novo código:

```
# Limpar ruído
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, np.ones((5,5), np.uint8))
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, np.ones((5,5), np.uint8))

# Encontrar contornos
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Desenhar contornos
cv2.drawContours(img_rgb, contours, -1, (255, 0, 0), 2)
```

Segmentação de imagem na prática

Exemplo de outras cores:

Cor	Limite aproximado
Vermelho	0–10 e 170–180
Laranja	10–25
Amarelo	20–35
Verde	35–85
Ciano	85–100
Azul	100–130
Roxo	130–160

Segmentação de imagem na prática

Uma observação:

HSV é um modelo de cores que representa uma cor de forma mais próxima de como os humanos a percebem do que o modelo RGB.

Enquanto no **RGB** uma cor é definida pelas intensidades de **Vermelho (Red)**, **Verde (Green)** e **Azul (Blue)**, no **HSV** ela é definida por:

- **H (Hue)** – Matriz: **qual é a cor.**
- **S (Saturation)** – Saturação: **quão intensa ou "pura" é a cor.**
- **V (Value)** – Valor (ou brilho): **quão clara ou escura é a cor.**

Exercício 1:

Selecione uma imagem e execute o código acima, realizando a mudança necessária de cor.

O aumento de dados é uma técnica para aumentar artificialmente o conjunto de treinamento, criando cópias modificadas de um conjunto de dados usando os dados existentes.

Isso inclui fazer pequenas alterações no conjunto de dados ou usar a aprendizagem profunda para gerar novos pontos de dados.

O aumento de dados e a geração de dados sintéticos são técnicas distintas, porém complementares, no aprendizado de máquina:

- **Dados aumentados:** envolve a criação de versões modificadas de dados existentes para aumentar a diversidade do conjunto de dados.

Por exemplo, no processamento de imagens, a aplicação de transformações como rotações, inversões ou ajustes de cores em imagens existentes pode ajudar os modelos a se generalizarem melhor.

- **Dados sintéticos:** se refere a dados gerados artificialmente, o que permite que pesquisadores e desenvolvedores testem e aprimorem algoritmos sem arriscar a privacidade ou a segurança dos dados do mundo real.

As técnicas de aumento não se limitam a imagens. Você também pode aumentar o áudio, o vídeo ou o texto.

Por que o aumento de dados é importante?

O aumento de dados ajuda os modelos de aprendizado de máquina a ter um desempenho melhor, aproveitando ao máximo os dados existentes.

Ele evita o excesso de ajuste, melhora a precisão e cria diversidade nos dados de treinamento, o que é crucial quando os conjuntos de dados são pequenos ou desequilibrados.

Ao simular variações do mundo real, ele torna os modelos mais robustos e confiáveis, sem a necessidade de coletar dados caros.

Em resumo, é uma maneira simples e eficiente de ajudar os modelos a aprender e generalizar de forma eficaz.

Quando você deve usar o aumento de dados?

- Para evitar que os modelos se **ajustem demais**.
- O conjunto de treinamento inicial é muito pequeno.
- Para melhorar a precisão do modelo.
- Reduzir o custo operacional de rotulagem e limpeza do conjunto de dados brutos.

Transformações geométricas: inverta, corte, gire, estique e amplie imagens aleatoriamente.

Você precisa ter cuidado ao aplicar várias transformações nas mesmas imagens, pois isso pode reduzir o desempenho do modelo.

Transformações do espaço de cores: altere aleatoriamente os canais de cores RGB, o contraste e o brilho.

Filtros: alteram aleatoriamente a nitidez ou o desfoque da imagem.

Apagamento aleatório: apaga uma parte da imagem inicial.

Mistura de imagens: mescla e mistura de várias imagens.

Técnicas avançadas:

- **Redes adversárias generativas (GANs):** usadas para gerar novos pontos de dados ou imagens.
- **Transferência de estilo neural:** uma série de camadas convolucionais treinadas para desconstruir imagens e separar contexto e estilo.

Exemplo 1: Classificação de Pneumonia:

Você pode usar o corte aleatório, o zoom, o alongamento e a transformação do espaço de cores para melhorar o desempenho do modelo.

No entanto, você precisa ter cuidado com certos aumentos, pois eles podem gerar resultados opostos.

Por exemplo, a rotação aleatória e a reflexão ao longo do eixo x **não são recomendadas para o conjunto de dados de imagens de raios X.**

Exemplo 1:
Classificação
de
Pneumonia:
Imagens reais
de raio-x

PNEUMONIA



COVID19



PNEUMONIA



NORMAL



COVID19



PNEUMONIA



PNEUMONIA



PNEUMONIA



PNEUMONIA



PNEUMONIA



NORMAL



PNEUMONIA



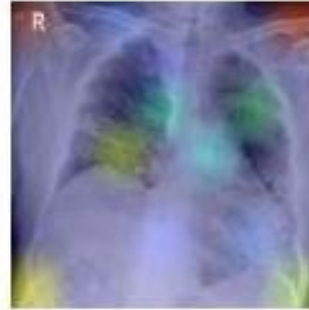
Exemplos reais

Exemplo 1: Classificação de Pneumonia: Após o aumento de dados inicial

PNEUMONIA pred as: PNEUMONIA



COVID19 pred as: COVID19



PNEUMONIA pred as: PNEUMONIA



NORMAL pred as: PNEUMONIA



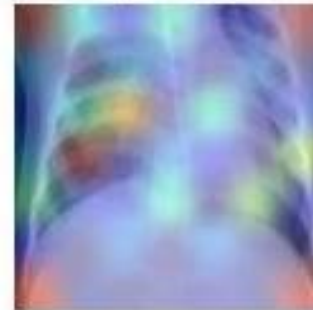
COVID19 pred as: COVID19



PNEUMONIA pred as: PNEUMONIA



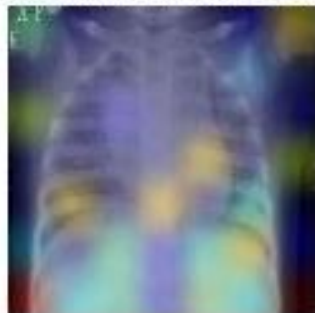
PNEUMONIA pred as: PNEUMONIA



PNEUMONIA pred as: PNEUMONIA



PNEUMONIA pred as: PNEUMONIA



PNEUMONIA pred as: PNEUMONIA



NORMAL pred as: NORMAL



PNEUMONIA pred as: PNEUMONIA



Exemplo 1: Classificação de Pneumonia:

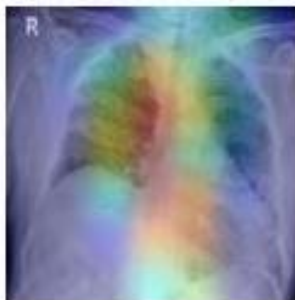
Após um
novo aument
o

Precisão de
95%

PNEUMONIA pred as: PNEUMONIA



COVID19 pred as: COVID19



PNEUMONIA pred as: PNEUMONIA



NORMAL pred as: NORMAL



COVID19 pred as: COVID19



PNEUMONIA pred as: PNEUMONIA



PNEUMONIA pred as: PNEUMONIA



PNEUMONIA pred as: PNEUMONIA



PNEUMONIA pred as: PNEUMONIA



PNEUMONIA pred as: PNEUMONIA



NORMAL pred as: NORMAL



PNEUMONIA pred as: PNEUMONIA



Exemplo 2: Carros autônomos:

Há poucos dados disponíveis sobre carros autônomos, e as empresas estão usando ambientes simulados para gerar dados sintéticos usando o aprendizado por reforço.



A biblioteca keras possui uma classe que facilita muito o nosso trabalho na hora de gerar novas imagens para alimentar o modelo.

Vamos supor que eu quisesse treinar uma CNN para classificar aviões militares, dentre elas o T-27 Tucano:



Para melhorar a quantidade de T-27 do nosso conjunto de treino e para generalizar mais o modelo de classificação, vamos gerar 10 novas imagens.

Inserir o código:

```
# importar os pacotes necessários
import numpy as np
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import os

# definir caminhos da imagem original e diretório do output
IMAGE_PATH = "/content/drive/MyDrive/Visao/t27_tucano.jpg"
OUTPUT_PATH = "/content//drive/MyDrive/Visao/output/"

# Criar o diretório de saída se não existir
os.makedirs(OUTPUT_PATH, exist_ok=True)

# carregar a imagem original e converter em array
image = load_img(IMAGE_PATH)
image = img_to_array(image)

# adicionar uma dimensão extra no array
image = np.expand_dims(image, axis=0)
```

Parte 2:

```
# criar um gerador (generator) com as imagens do data augmentation
imgAug = ImageDataGenerator(rotation_range=45, width_shift_range=0.1,
                             height_shift_range=0.1, zoom_range=0.25,
                             fill_mode='nearest', horizontal_flip=True)

imgGen = imgAug.flow(image, save_to_dir=OUTPUT_PATH,
                     save_format='jpg', save_prefix='t27_')

# gerar 10 imagens por data augmentation
counter = 0

for (i, newImage) in enumerate(imgGen):
    counter += 1
    # ao gerar 10 imagens, parar o loop
    if counter == 10:
        break
```

As 10 imagens geradas estarão na pasta de saída:

`"/content/drive/MyDrive/Visao/output/"`



Exercício 2:

Selecione uma nova imagem e repita o código.

Quantização é uma técnica de otimização que consiste em **reduzir a precisão numérica dos pesos e das ativações de um modelo de inteligência artificial**, diminuindo o consumo de memória e acelerando a inferência, especialmente em dispositivos com recursos limitados (dispositivos de borda).

A quantização e otimização para dispositivos de borda em visão computacional consistem em adaptar modelos de IA para executar em hardware com recursos limitados, como câmeras inteligentes, smartphones, drones, Raspberry e microcontroladores, reduzindo consumo de memória, latência e energia sem comprometer excessivamente a acurácia.

Tipo	Precisão	Vantagens	Desvantagens
FP32	32 bits	Máxima precisão	Alto consumo.
FP16	16 bits	Boa velocidade em GPUs	Pequena perda de precisão.
INT8	8 bits	Até 4× menos memória e maior velocidade	Pequena perda de acurácia.
INT4	4 bits	Muito compacto	Pode degradar significativamente o desempenho.
Binary	1 bit	Extremamente leve	Aplicações bastante específicas.

Como funciona?

Durante o treinamento, a maioria dos modelos utiliza números em **ponto flutuante de 32 bits (FP32)**. Esses números representam valores com alta precisão, mas exigem mais memória e maior capacidade computacional.

Exemplo: **0,72846395 (FP32)**

Na quantização para INT8, esse valor é convertido para um inteiro de 8 bits, juntamente com uma escala que permite reconstruir aproximadamente o valor original:

FP32	→	INT8
0,72846395	→	74

Durante a inferência, o hardware utiliza o inteiro e a escala para realizar os cálculos de forma muito mais eficiente.

Por que isso melhora o desempenho?

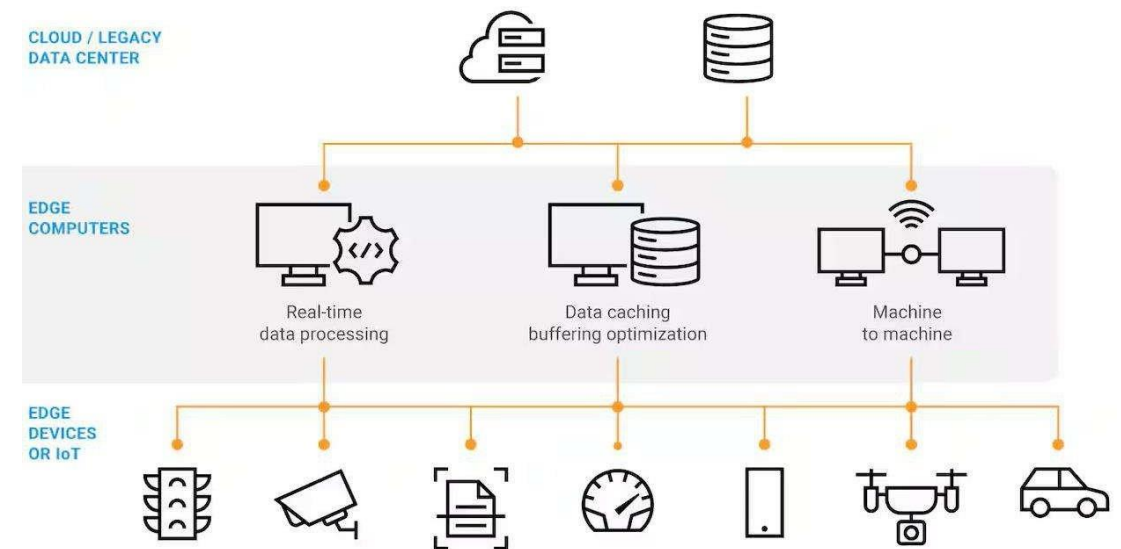
A redução da precisão traz três benefícios principais:

- **Menor uso de memória:** um peso em FP32 ocupa 32 bits, enquanto um peso em INT8 ocupa apenas 8 bits, reduzindo o tamanho do modelo em cerca de 75%.
- **Inferência mais rápida:** muitos processadores possuem instruções específicas para operações com inteiros de 8 bits.
- **Menor consumo de energia:** menos dados são movimentados na memória e as operações aritméticas são mais simples.

Aplicações:

A quantização é amplamente utilizada em sistemas que executam modelos de IA diretamente no dispositivo, como:

- smartphones;
- câmeras inteligentes;
- drones;
- robôs móveis;
- sistemas de monitoramento;
- dispositivos IoT;
- veículos autônomos.



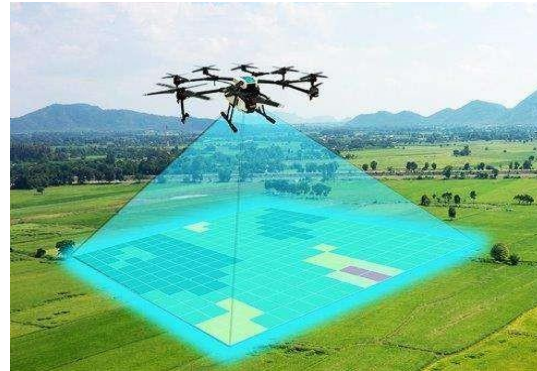
Nesses cenários, reduzir memória, tempo de inferência e consumo de energia é essencial.

Quantização e otimização para dispositivos de borda

O processamento na borda possibilita executar modelos de IA diretamente nos dispositivos, incluindo nós, próximos à fonte de dados.

Exemplos de aplicações:

- Monitoramento de saúde com dispositivos vestíveis (smartwatches e pulseiras fitness.)
- Automação agrícola com drones.
- Detecção de falhas em linhas de produção.
- Aumento do desempenho, eficiência energética e robustez de redes móveis.



Limitações dos dispositivos de borda



Porém, em geral, estes dispositivos apresentam algumas limitações:

- Alimentação via bateria ou limitada.
- Baixa capacidade de processamento.
- Armazenamento limitado.

Entretanto, essas limitações são as motivações para o desenvolvimento de modelos mais eficientes.

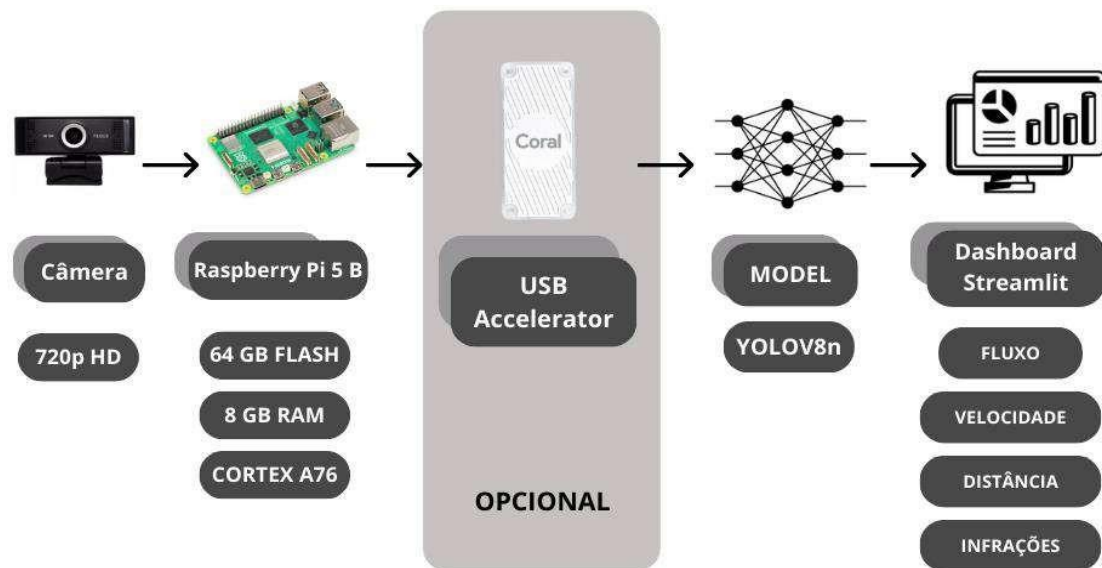
Exemplo: Monitoramento de veículos



- **Problema:** o aumento do volume e da complexidade do tráfego torna o monitoramento de veículos uma tarefa desafiadora para controladores e agentes de trânsito.
- **Proposta:** desenvolver uma solução para o monitoramento autônomo de veículos em tempo real, reduzindo a carga de operadores e agentes de trânsito.

Quantização e otimização para dispositivos de borda

Exemplo: Monitoramento de veículos



- A solução usa uma câmera HD 720p, uma Raspberry Pi 5 (ARM Cortex-A76 quad-core 64-bit @2.4GHz com 8 GB de RAM).
- O YOLOv8n é usado para rastrear objetos (i.e., veículos e pessoas).
- A solução analisa autonomamente e em tempo real o fluxo, velocidade e distância de objetos.
- Gera notificações de excesso de velocidade, incluindo as placas, e alertas de colisões eminentes.

Quantização e otimização para dispositivos de borda

Métricas de desempenho

Versão	mAP0.5 (%)		P (%)	R (%)	F1	Size (MB)
float32	74.2		71.5	69.9	70.7	9
float16	74.0		70.0	69.0	70.0	5
int8	70.0		70.0	68.5	69.5	3

Exemplo: Monitoramento de veículos

Modelo rodando na CPU

Versão	FPS	CPU (%)	RAM (GB)
float32	15	20	0.9
float16	16	15	0.7
int8	17	12	0.5

Modelo rodando na Edge TPU

Versão	FPS	CPU (%)	RAM (GB)
float32	26	17	0.7
float16	32	15	0.6
int8	35	10	0.3

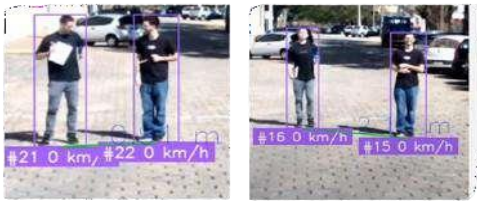
Resultado:
O modelo opera em tempo real, mantém bom desempenho mesmo quantizado e consome poucos recursos (CPU, RAM e Memória).

Testes de velocidade e distância



Medição: 30km/h
Resultado: 28km/h

Medição: 40km/h
Resultado: 36km/h



Medição: 1 m
Resultado: 0.91 m

Medição: 2 m
Resultado: 2.11 m

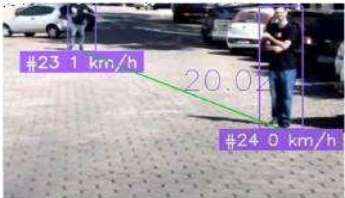


Medição: 20km/h
Resultado: 20 km/h

Medição: 10km/h
Resultado: 10km/h



Medição: 5 m
Resultado: 4.95 m



Medição: 20 m
Resultado: 20.02 m

Vamos para a prática:

```
#Executando o rastreamento de objetos
results = model.track(
    source="/content/drive/MyDrive/Visao/transito.mp4",
    tracker="bytetrack.yaml",
    persist=True
)
```

Quantização e otimização para dispositivos de borda

Novo Código:

```
# Esse código exporta o modelo YOLOv8 do formato PyTorch (.pt) para o formato ONNX (.onnx).  
model.export(  
    format="onnx",  
    imgsz=640  
)
```

Quantização e otimização para dispositivos de borda

Novo Código:

```
# Esse comando instala as bibliotecas necessárias para executar e/ou otimizar modelos ONNX  
!pip install -q onnxruntime onnxruntime-tools
```

Novo Código:

```
# Função de quantização
from onnxruntime.quantization import quantize_dynamic, QuantType

quantize_dynamic(
    model_input="/content/yolov8n.onnx",
    model_output="/content/yolov8n_int8.onnx",
    weight_type=QuantType.QInt8
)

print("Modelo quantizado salvo!")
```

Quantização e otimização para dispositivos de borda

Novo Código:

```
# Verifica o tamanho dos arquivos
#yolov8n.pt Modelo original em PyTorch (FP32)
#yolov8n.onnx Modelo exportado para ONNX (FP32)
#yolov8n_int8.onnx Modelo ONNX quantizado (INT8)

import os

for f in [
    "/content/yolov8n.pt",
    "/content/yolov8n.onnx",
    "/content/yolov8n_int8.onnx"
]:
    print(f, os.path.getsize(f)/1024/1024, "MB")
```

Detectando as placas:

```
!pip install ultralytics easyocr opencv-python -q
```

Detectando as placas:

```
import cv2  
import easyocr  
from ultralytics import YOLO
```

Quantização e otimização para dispositivos de borda

Detectando as placas:

```
def detect_plates(video_path):  
  
    cap = cv2.VideoCapture(video_path)  
  
    results_data = []  
  
    while cap.isOpened():  
        ret, frame = cap.read()  
        if not ret:  
            break  
  
        detections = model(frame)[0]  
  
        for box in detections.bboxes:  
            cls = int(box.cls[0])  
  
            # classes COCO: 2=car, 3=moto, 7=truck, 5=bus  
            if cls in [2, 3, 5, 7]:  
  
                x1, y1, x2, y2 = map(int, box.xyxy[0])  
  
                vehicle_crop = frame[y1:y2, x1:x2]
```

Quantização e otimização para dispositivos de borda

Parte 2:

```
# região inferior (onde geralmente fica a placa)
h = vehicle_crop.shape[0]
plate_region = vehicle_crop[int(h*0.6):h, :]

# OCR
ocr_result = reader.readtext(plate_region)

plate_text = ""

for r in ocr_result:
    plate_text += r[1] + " "

plate_text = plate_text.strip()

if plate_text != "":
    results_data.append(
        {
            "plate": plate_text,
            "bbox": (x1, y1, x2, y2)
        }
    )

print("Placa detectada:", plate_text)
```


Parte 3:

```
cap.release()
```

```
return results_data
```

Impressão das placas detectadas:

```
results = detect_plates("/content/drive/MyDrive/Visao/transito.mp4")
```

Quantização e otimização para dispositivos de borda

Exercício 3:

Encontrem um outro vídeo de trânsito e repita os códigos.